

LABORATORIO 4: SSH Y TEMPORIZADORES

| Parte 1: SSH

Conceptos Teóricos:

SSH proviene de "Secure Shell" (Intérprete Seguro) y es un protocolo de administración remota que permite el control de un dispositivo remoto.

Diagrama SSH Client y SSH Server

SSH necesita un nombre de host/IP, un puerto, credenciales de usuario y contraseña.

Necesitamos habilitar SSH en nuestra Raspberry Pi:

Menú Preferencias Raspberry Pi

Configuración de Interfaces Raspberry Pi

Después de habilitar SSH en la Raspberry Pi, podemos acceder a este dispositivo de forma remota utilizando su dirección IP y la contraseña de usuario. Para ello, se requiere una red común entre la Raspberry y la computadora.

Podemos seguir los siguientes pasos para conectarnos y desconectarnos usando SSH:

1. Habilitar el SSH de la Raspberry Pi desde Configuración de Raspberry Pi -> Interfaces -> Habilitar SSH.
2. Se recomienda cambiar la contraseña de la Raspberry usando el comando `sudo raspi-config`.
3. La IP de la Raspberry Pi se puede mostrar usando el comando `hostname -I`.
4. Utilice el comando `ssh pi@[IP_raspberry]` en Ubuntu, por ejemplo: `ssh pi@111.111.1.111`.
5. Por lo general, después del paso 4 es necesario introducir la contraseña y el nombre de usuario de la Raspberry.
6. Para finalizar la comunicación SSH, escriba `exit` en la terminal.

¿Cuál es el puerto predeterminado que utiliza SSH?

¿Podemos conectarnos a nuestra Raspberry Pi utilizando dispositivos diferentes a nuestras computadoras, como nuestros teléfonos inteligentes? Si es así, ¿cómo podemos hacerlo?

Ejercicios:

- Usando SSH, cree una nueva carpeta en su Raspberry Pi con el nombre lab4, luego cree un archivo de Python dentro de la carpeta con el nombre buzzer.py.
- Genere un programa en Python en su computadora que haga sonar un zumbador (buzzer) cuando se presione un botón.
- Usando SSH, modifique el programa anterior para que el zumbador suene cuando se presione un botón y se detenga cuando se presione un segundo botón.
- Cree una secuencia de LEDs que tenga un intervalo de tiempo variable entre estados.
- Haga el programa de manera que pueda recibir modificaciones por SSH sin detener la ejecución.
Consejo: Modifique un archivo txt con SSH y lea los valores del archivo en el programa principal desde la Raspberry Pi.
- ¿Puede programar en Python usando SSH desde su computadora?
- Investigue cómo VSCode puede lograr trabajar de forma remota usando la extensión 'Remote-SSH'.

Parte 2: Temporizadores (Timers)

Los temporizadores se utilizan para generar operaciones de acuerdo con ciertos eventos durante el tiempo.

Generalmente, generaremos interrupciones en la rutina principal cuando se ejecuta un temporizador.

- Podemos usar retardos (delays) para interrumpir toda la ejecución del programa y temporizadores para cambiar la rutina principal por una subrutina.

```
uint32_t FS = 120000000*2;
void timer0A_handler(void);
uint8_t switch_state = 0;
uint32_t button = 0;
int main(void)
{
    // Configuración del Reloj
    SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ | SYSCTL_OSC_MAIN | SYSCTL_USE_PLL |
SYSCTL_CFG_VCO_480), 120000000); // Habilita el reloj del sistema
    // Habilitar periféricos
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION); // Habilita periférico N
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOF); // Habilita periférico F

    // Habilitar Periférico Temporizador
    SysCtlPeripheralEnable(SYSCTL_PERIPH_TIMER0);
```

Fragmento de código C Tiva Timer

Debemos configurar el temporizador para que funcione correctamente. Para ello, es necesario definir una Frecuencia de Muestreo (Frequency Sample o FS) que actúe como preescalador para nuestro temporizador. La siguiente fórmula puede ayudarnos a seleccionar el FS adecuado:

$$timerFreq = clockfreq / FS$$

Con el FS seleccionado, debemos iniciar la interrupción del procesador y luego habilitar el temporizador:

```
// Configurar temporizador
TimerConfigure(TIMER0_BASE, TIMER_CFG_PERIODIC);
// Configurar el tiempo de conteo para el Temporizador
TimerLoadSet(TIMER0_BASE, TIMER_A, FS);
// Habilitar interrupciones del procesador
IntMasterEnable();
// Habilitar Interrupción
IntEnable(INT_TIMER0A);
// Habilitar interrupción del temporizador A
TimerIntEnable(TIMER0_BASE, TIMER_TIMA_TIMEOUT);
// Habilitar el temporizador
TimerEnable(TIMER0_BASE, TIMER_A);
```

El temporizador utiliza una función externa para ejecutar la subrutina. Esta rutina detendrá la rutina principal y, dependiendo de si el temporizador es periódico o de disparo único (one shot), continuará ejecutándose.

```

void timer0A_handler(void)
{
    switch_state++;
    // Cambiando el retardo del temporizador
    if (GPIOPinRead(GPIO_PORTJ_BASE, GPIO_PIN_0) == 0)
    {
        button++;
        if(button > 0)
        {
            FS = 120000000*4;
        }
        TimerLoadSet(TIMER0_BASE, TIMER_A, FS);
    }
    // Limpiar temporizador
    TimerIntClear(TIMER0_BASE, TIMER_A);

    // Secuencia 1.
    if(switch_state % 2 != 0)
    {
        GPIOPinWrite(GPIO_PORTN_BASE, 0x03, 0x02); // establecer base periférica,
pines, valores
        GPIOPinWrite(GPIO_PORTF_BASE, 0x11, 0x01); // establecer base periférica,
pines, valores
    }
    // Secuencia 2.
    if(switch_state % 2 == 0)
    {
        GPIOPinWrite(GPIO_PORTN_BASE, 0x03, 0x01);
    }
}

```

Debemos declarar la función del temporizador en el archivo de inicio (startup) del proyecto.

StartUp

```
//*****  
//  
// El punto de entrada para la aplicación.  
//  
//*****  
extern int main(void);  
extern void timer0A_handler(void);  
//*****  
    IntDefaultHandler,    // Secuencia ADC 0  
    IntDefaultHandler,    // Secuencia ADC 1  
    IntDefaultHandler,    // Secuencia ADC 2  
    IntDefaultHandler,    // Secuencia ADC 3  
    IntDefaultHandler,    // Temporizador Watchdog  
    timer0A_handler,      // Temporizador 0 subtimer A  
    IntDefaultHandler,    // Temporizador 0 subtimer B  
    IntDefaultHandler,    // Temporizador 1 subtimer A  
    IntDefaultHandler,    // Temporizador 1 subtimer B
```

Este proceso nos ayudará a generar una secuencia de temporizador y utilizarla para realizar una interrupción en el proceso principal.

Ejercicios:

- Hacer parpadear el LED de usuario PN1 con interrupciones de temporizador en la Tiva. Inicialmente, hacer parpadear el LED cada segundo, luego modificar el código para hacerlo parpadear cada 2 segundos y finalmente cambiar el código para hacerlo parpadear cada 5 segundos.
- Hacer una secuencia decimal con los LEDs de usuario que cambie cada 2 segundos.
- Modificar el ejercicio anterior para cambiar el intervalo a 1 y 0,5 segundos cuando se presiona un interruptor de usuario.
- Modificar el ejercicio de contador binario del laboratorio 3 para usar temporizadores; cada secuencia debe cambiar sin botones y en un intervalo de 1.5 segundos. Luego agregue una nueva funcionalidad: cuando se presione un botón, cambie el intervalo de tiempo a 3 segundos.
- Para el siguiente ejercicio, generará una secuencia de 3 estados con los LEDs de usuario de la TIVA. Esta secuencia debe funcionar usando un temporizador y, mediante SSH, debe poder cambiar los segundos en que funciona el temporizador. Para esto debe:
 - Crear una secuencia de 3 estados en TIVA.
 - Implementar la secuencia en un Temporizador.
 - Usando SSH, cambiar los segundos que tarda el temporizador en generar la secuencia.
- ¿Cómo se puede implementar un temporizador que funcione una sola vez? Investigue sobre los temporizadores de disparo único (one shot timers).